Neighborhood Library Rubric

Your project is scored on five outcomes. If you complete every requirement in the instructions, you are Meeting. To reach Exceeding, add the extras described in that column. Use this as a checklist while you build.

Outcome	Beginning	Approaching	Meeting	Exceeding
Model data with a class	The Book class is missing, or it only has a few of the required properties.	The Book class exists but is incomplete. Some getters or setters are missing, a property has the wrong type, or checkOut() and checkIn() only update one of the two properties they should change.	The Book class has all five properties as private fields with the correct types, a constructor, and getters and setters. checkOut(name) sets checkedOutTo to the name and isCheckedOut to true. checkIn() sets checkedOutTo to "" and isCheckedOut to false.	The Book class protects its own data. For example, checkOut() refuses to check out a book that is already checked out, the id has no setter so it can't be changed, or a toString() method formats the book for display.
Store data in an array	There is no array of books, or the array only holds a few books.	The array exists but has fewer than 20 books, some books share the same id, or the Available and Checked Out screens show the wrong books.	The array holds at least 20 books, each with a unique id. The Available Books screen shows only books that are not checked out, the Checked Out Books screen shows only books that are, and the program can find a book using its id.	Code you would otherwise repeat, like looping through the array to find a book by id, is written once in its own method and reused. When a list is empty, the user sees a message such as "No books are currently checked out" instead of a blank screen.
Control the menu flow	The program runs once and ends, or choosing a menu option doesn't lead to the right screen.	The home screen works, but at least one screen is missing, a screen doesn't return to the home screen when it should, or Exit doesn't close the program.	The home screen keeps showing until the user chooses Exit. Every screen works the way the instructions describe and returns to the home screen when it's done.	Each screen is written as its own method, so main stays short and easy to read. The user can never get stuck on a screen with no way back.
Respond to user input	The program ignores what the user types, or the user can't move between screens.	Correct input mostly works, but some prompts get skipped (a common issue when mixing nextInt() and nextLine()), or entering an id that doesn't exist causes wrong behavior.	Correct input works on every screen. Every prompt tells the user exactly what to type. After a book is checked out or checked in, the program shows a message confirming it.	The program uses if statements to catch mistakes and show the user a clear message. This includes an id that isn't in the inventory, checking out a book that's already checked out, checking in a book that isn't checked out, and typing a lowercase c or x.
Write readable code	The code is hard to follow. Indentation is inconsistent and variables have names like x, b, or temp that don't explain what they hold.	The code is mostly readable, but a few variable names are vague or the formatting is uneven. The Book class may be in the same file as main.	The Book class is in its own file inside a named package. Variable names describe what they hold, like selectedId or userName. The code is formatted consistently (IntelliJ's Reformat Code is fine to use).	Another student could read main and follow how the program works quickly, because each method has a clear name that says what it does, like showAvailableBooks() or checkInBook().